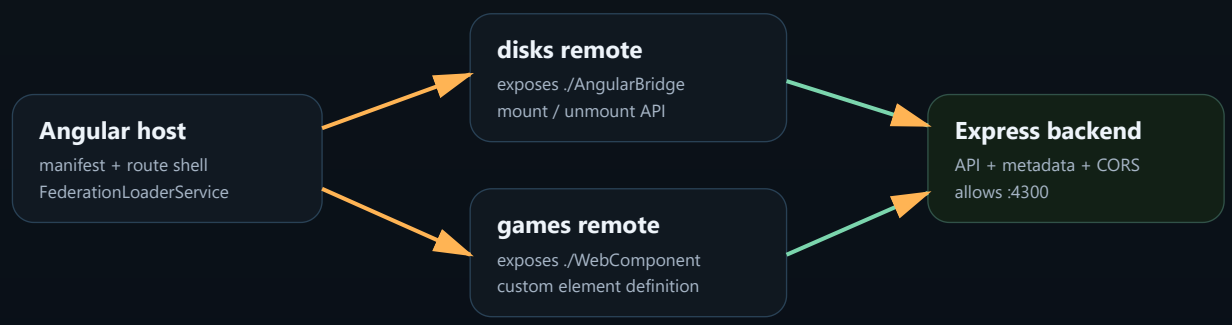
Tutorial: Load React Micro Front Ends from an Angular Host

This tutorial documents the working path used in the FS64 workspace to load two React MFEs from Angular: `disks` through a React bridge and `games` through a Web Component wrapper. It omits the dead ends and keeps only the steps that matter.

Target result

An Angular host on `:4300` loads React remotes dynamically from a manifest, mounts `disks` inside an Angular view via an imperative bridge, and renders `games` via a custom element.

1. Final Architecture



2. Prerequisites

Workspace state

- Nx workspace with React remotes already running under Rspack.
- Angular support installed through Nx.
- Shared registry/runtime helper available in a workspace library.

Ports used in this walkthrough Angular host: 4300 | disks: 4201 | games: 4202 | backend: 5000

3. Step-by-Step

1 Create an Angular host app

Create a standalone Angular application in the Nx workspace. The host does not need to own business logic. It only needs to:

- load a remote manifest,
- resolve remotes by id,
- render routed demo pages.

```
npx nx g @nx/angular:application angular-host \
  --bundler=rspack \
  --routing \
  --standalone \
```

```
  --style=css \
  --port=4300
```

2 Expose a React bridge from the disks remote

The `disks` MFE is a full React slice. Angular should not try to import it as a component directly. Instead, expose an imperative bridge module that mounts the React app into a host DOM element.

```
// apps/disks/module-federation.config.ts
exposes: {
  './Module': './src/remote-entry.ts',
  './AngularBridge': './src/angular-bridge.tsx',
}
```

```
// apps/disks/src/angular-bridge.tsx
export function mountFs64DisksBridge(element: Element, options?: { initialPath?: string }): void {
  const root = createRoot(element);
  root.render(
    <MemoryRouter initialEntries={[options?.initialPath || '/']}>
      <App />
    </MemoryRouter>
  );
}

export function unmountFs64DisksBridge(element: Element): void {
  // tear down root
}
```

Reason: this pattern gives Angular explicit mount and unmount control for a large embedded React app.

3 Expose a Web Component wrapper from the games remote

The `games` slice also works well as a widget. For that case, a custom element is simpler than an imperative bridge.

```
// apps/games/module-federation.config.ts
exposes: {
  './Module': './src/remote-entry.ts',
  './WebComponent': './src/web-component.tsx',
}
```

```
// apps/games/src/web-component.tsx
class Fs64GamesElement extends HTMLElement {
  static get observedAttributes() {
    return ['query'];
  }

  private renderReactTree() {
    const query = this.getAttribute('query') || '';
    const initialPath = query ? `/?q=${encodeURIComponent(query)}` : '/';

    this.root.render(
      <MemoryRouter initialEntries={[initialPath]}>
        <App />
      </MemoryRouter>
    );
  }
}

export function defineFs64GamesElement(tagName = 'fs64-games-widget'): void {
  if (!customElements.get(tagName)) {
    customElements.define(tagName, Fs64GamesElement);
  }
}
```

Reason: Angular can treat the remote like plain HTML once the element is registered.

4 Use a shared manifest-driven loader in Angular

Do not hardcode remote URLs directly inside Angular page components. Put them in a manifest and resolve by id.

```
// angular-host/public/fs64-remotes.json
{
  "remotes": [
    { "id": "disks", "entry": "http://localhost:4201/remoteEntry.js", "route": "/disks" },
    { "id": "games", "entry": "http://localhost:4202/remoteEntry.js", "route": "/games" }
  ]
}
```

```
// angular-host/src/app/federation-loader.service.ts
@Injectable({ providedIn: 'root' })
export class FederationLoaderService {
  async loadRemoteById<T>(id: string, exposedModule = './Module'): Promise<T> {
    const remote = await this.getRemoteById(id, exposedModule);
    return loadFs64RemoteModule<T>(remote);
  }
}
```

5 Create Angular pages for each interop style

Keep each demo page focused on one integration pattern.

```
// Disks bridge page
async ngAfterViewInit() {
  this.bridge = await this.loader.loadRemoteById<DisksBridgeModule>('disks', './AngularBridge');
  this.bridge.mountFs64DisksBridge(this.mountPoint.nativeElement, {
    initialPath: '/?diskId=1',
  });
}
```

```
// Games web-component page
async ngAfterViewInit() {
  const module = await this.loader.loadRemoteById<GamesElementModule>('games', './WebComponent');
  module.defineFs64GamesElement();
}
```

```
<fs64-games-widget [attr.query]="query"></fs64-games-widget>
```

6 Make the Angular host a real federation host

This is the critical step. A plain Angular/Rspack app is not enough. The React remotes expect a federation host runtime.

```
// angular-host/module-federation.config.ts
const config: ModuleFederationConfig = {
  name: 'angular-host',
  remotes: [],
};

export default config;
```

```
// angular-host/rspack.config.ts
import { NxModuleFederationPlugin } from '@nx/module-federation/src/plugins/nx-module-federation-plugin/angular/nx-module-federation-
plugin';
import { NxModuleFederationDevServerPlugin } from '@nx/module-federation/src/plugins/nx-module-federation-plugin/angular/nx-module-
```

```
federation-dev-server-plugin';
import config from './module-federation.config';

rspackConfig.plugins.push(new NxModuleFederationPlugin({ config }, { dts: false }));
rspackConfig.plugins.push(new NxModuleFederationDevServerPlugin({ config }));
```

Reason: without a real federation host runtime, the remotes fail during share-scope initialization.

7 Make the shared loader tolerant for this interop experiment

In this setup, Angular is an interop host first and a fully standardized federation host second. The shared runtime loader should tolerate missing or partial share-init behavior and still initialize the container.

```
async function tryInitHostShareScope(): Promise<void> {
  if (typeof __webpack_init_sharing__ !== 'function') {
    return;
  }

  try {
    await __webpack_init_sharing__('default');
  } catch {
    // Continue with an empty share scope for the interop experiment.
  }
}

export async function loadFs64RemoteModule<T>(remote: Fs64ResolvedRemoteManifestEntry): Promise<T> {
  await tryInitHostShareScope();
  await loadRemoteEntry(remote);
  await container.init(__webpack_share_scopes__?.default ?? {});
  return factory() as T;
}
```

8 Allow Angular host origin in backend CORS

The remotes still fetch their own data from the backend. Once they are rendered inside Angular, the origin becomes `:4300`. If backend CORS does not allow that origin, you will see `Failed to fetch` even though the embedding works.

```
const allowedOrigins = new Set([
  'http://localhost:4200',
  'http://localhost:4201',
  'http://localhost:4202',
  'http://localhost:4300',
  'http://127.0.0.1:4200',
  'http://127.0.0.1:4201',
  'http://127.0.0.1:4202',
  'http://127.0.0.1:4300',
]);
```

9 Run the minimum process set

For this experiment, keep the process set small and explicit.

```
npm run dev:backend
npm run dev:angular-host
npm run dev:disks
npm run dev:games
```

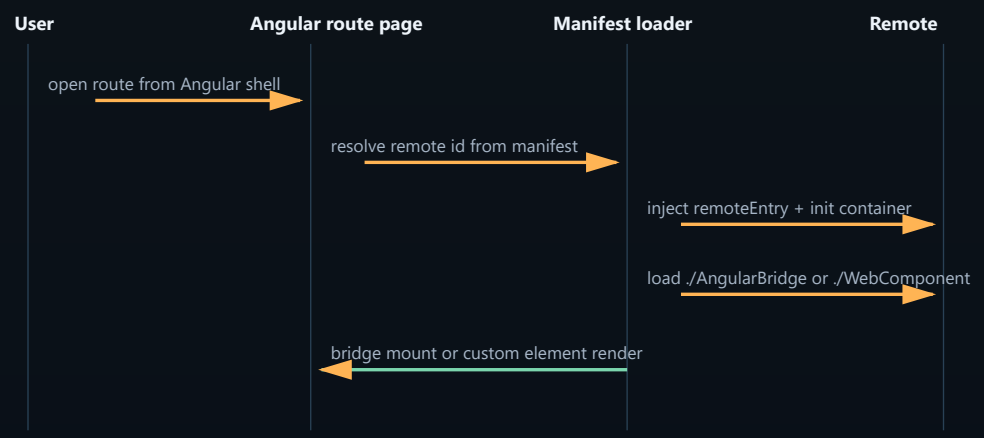
Then open:

```
http://localhost:4300/
```

Use the Angular host navigation to switch between:

- `Disks Bridge`
- `Games Web Component`

4. Relationship and Flow



5. Code Files Involved

File	Purpose
`apps/disks/src/angular-bridge.tsx`	Imperative React mount/unmount bridge for Angular.
`apps/games/src/web-component.tsx`	Custom element wrapper for the React games slice.
`angular-host/src/app/federation-loader.service.ts`	Manifest-driven remote resolution and dynamic module load.
`angular-host/src/app/disks-bridge-page.component.ts`	Angular page that mounts the disks bridge into a DOM node.
`angular-host/src/app/games-webcomponent-page.component.ts`	Angular page that registers and renders the games custom element.
`angular-host/module-federation.config.ts`	Turns Angular host into a federation host.
`angular-host/rspack.config.ts`	Adds the Angular-specific Nx federation plugins.
`libs/fs64/registry/src/lib/runtime.ts`	Shared dynamic remote loader used by the Angular host.
`apps/backend/server.js`	Adds CORS for `:4300` so embedded remotes can fetch data.

6. When to Use Each Pattern

Use a React bridge when

- the remote is a full app slice,
- you need explicit lifecycle control,
- the host should mount/unmount the remote manually.

Use a Web Component when

- the remote can behave like a widget,
- the host should interact through attributes or DOM events,
- you want a framework-neutral integration boundary.

7. Success Criteria

Working result

- Angular host opens at `http://localhost:4300/`.
- `Disks Bridge` renders the React `disks` slice inside Angular.
- `Games Web Component` renders the React `games` slice inside Angular.
- Both embedded remotes can fetch live data from the backend.

8. Recommended Follow-Up

- Move Angular host manifest loading to the backend manifest endpoint with local fallback.
- Stabilize Angular host deep-link behavior so route refreshes work cleanly in dev.
- Decide which interop style should be the standard for future non-React hosts.
- Add a third Angular page to test `play` with the same approach.

Tutorial generated from the working path implemented in the FS64 monorepo. This is the ﾟgo pathﾟ only.